

Kemika Sherchan
PRG 400
Week 3 Assignment

In asynchronous programming, a program can process several tasks simultaneously without completion of the tasks in a particular order. E.g. when a program is waiting to be fed with information (such as via the internet) it does not have to remain idle. Rather, it is able to put its hand to a different task and return later. Python supports the library of asynchronous Python, known as asynchronous. It uses concepts like:

- Event loop
- Async functions (async and await)
- Callbacks

They are useful in accelerating programs and making them more efficient particularly when it comes to delays.

Event Loop

The main aspect of asynchronous programming in python is the event loop. It will be like a manager that monitors all the tasks and at what time to execute them.

In lieu of running them one at a time, the event loop interchanges them in case one task is in waiting.

```
In [5]: import asyncio

# Async function
async def say_hello():
    print("Hello")
    await asyncio.sleep(1) # simulate delay
    print("World")

await say_hello()
```

Hello
World

Code Explanation

- An asynchronous function is defined by the use of the keyword, asynchronous def.
- await causes the suspension of the function and the running of other

functions.

- `await asyncio.sleep(1)` is another simulation of waiting.
- We replace use of the function, `await` with the use of the run method, which is an asynchronous method in Jupyter Notebook.

Program Flow

1. The function begins and is followed by the printing of Hello.
2. It pauses for 1 second.
3. Once there has been a delay, it will print World.
4. This execution is done through the event loop.

Asynchronous Programming (async / await)

In asynchronous programming, several tasks can run concurrently without the blocking of each other.

Python also dedicates tasks, unlike waiting until one task is complete, which is better utilization of time.

```
In [4]: import asyncio

async def task1():
    print("Task 1 started")
    await asyncio.sleep(2)
    print("Task 1 finished")

async def task2():
    print("Task 2 started")
    await asyncio.sleep(1)
    print("Task 2 finished")

async def main():
    await asyncio.gather(task1(), task2())

await main()
```

```
Task 1 started
Task 2 started
Task 2 finished
Task 1 finished
```

Code Explanation

- Two tasks are established, i.e., task1 and task2.
- Every task has a pause with awaiting asynchronous sleep with asynchronous sleep.
- `asyncio.gather()` is used to run both tasks simultaneously.

Program Flow

1. These two tasks are initiated nearly simultaneously.
2. Task 2 completes the first (1 second after).
3. Task 1 is completed later (after 2 seconds).
4. This demonstrates tasks that are executed simultaneously as opposed to sequentially.

Callbacks

A callback is a function which is automatically invoked once some other task has been completed.

It is useful when we want to perform an action once a task finishes.

```
In [3]: import asyncio

# Callback function
def callback_function(task):
    print("Callback: Task completed!")

async def my_task():
    print("Task started")
    await asyncio.sleep(2)
    print("Task finished")

async def main():
    task = asyncio.create_task(my_task())

    # Add callback
    task.add_done_callback(callback_function)

    await task

await main()
```

```
Task started
Task finished
Callback: Task completed!
```

Code Explanation

- `asyncio.create_task()` is used to create a background task.
- add done callback
- adds a function that will be executed when the task is done.
- Callback function will display a message when the task is done.

Program Flow

1. The task starts running.

2. It waits for 2 seconds.
3. The callback mechanism is automatically activated when completed.
4. The completion is displayed in a message.